Kasi Yedumati · Full Stack Training

Biweekly Assessment BW04 — Weeks 7–8

Sessions covered: S31–40
Topics: Inheritance (`extends`, `super`, override, IS-A); abstract classes; interfaces (`implements`); polymorphism; packages; `List` / `ArrayList`, `Set` / `HashSet`, `Map` / `HashMap`

Timebox: 45–60 minutes (homework or Friday of the even week).
Rules: Closed notes unless your trainer allows open notes. No copying.
Coding constraint: `extends` / `abstract` / `implements` and `List` / `Set` / `Map` are OK. **No `throw` / `try` / `catch`.** Validate with `print` + `return`. No Spring. No Lombok.

A. Multiple Choice (12)

Circle the best option.

1. `class Faculty extends Staff` means Faculty:
   a) IS-A Staff
   b) HAS-A Staff
   c) Is a package
   d) Must use Spring
2. `super(name)` in a child constructor:
   a) Must be the first statement
   b) Must be the last statement
   c) Is illegal
   d) Creates a second parent
3. `@Override void display()` in a child:
   a) Same signature as the parent method, runtime object chooses the body
   b) A different parameter list (overloading)
   c) Only works on `static` methods
   d) Deletes the parent class
4. A Java class can `extends`:
   a) Many classes at once
   b) **One** class
   c) Only interfaces
   d) Only `String`
5. `abstract class Account` means:
   a) You can `new Account(...)` anywhere
   b) You **cannot** `new Account(...)`; children fill in abstract methods
   c) It cannot have a constructor
   d) It cannot have fields
6. `class WalletAccount implements Payable` is legal because `Payable` is:
   a) A class parent
   b) An **interface** (capability / contract)
   c) A primitive
   d) A package only

7. Polymorphism means:
   a) Same call site (`a.kind()`, `p.pay(amount)`), different runtime objects
   b) Two files with the same name
   c) Only `static` methods
   d) Skipping `super`
8. `package campus.wallet;` must match:
   a) A random folder
   b) Folder `campus/wallet/`
   c) Only the `.class` file name
   d) MySQL schema names
9. `List<String> cart = new ArrayList<>();` is preferred because:
   a) Arrays are illegal now
   b) You **code to the interface** (`List`) and construct `ArrayList`
   c) `HashSet` cannot hold strings
   d) Generics are optional forever
10. `HashSet.add("P101")` when `"P101"` is already in the set:
    a) Stores a second copy
    b) Returns `false` and keeps size the same
    c) Crashes
    d) Converts the set into a List
11. `map.get("missing")` on a `HashMap` returns:
    a) `0`
    b) `""`
    c) `null`
    d) The first key
12. `int qty = stock.get("P999");` when `"P999"` is not a key (`Map<String, Integer>`):
    a) Prints 0 quietly
    b) Unboxes `null` → `NullPointerException`
    c) Compiles only with Spring
    d) Inserts 0 automatically

B. Coding (2)

1. Write `abstract class Account` with private `holder` and `balance`, a constructor, `getHolder()`, and `abstract String kind()`. `SavingsAccount` and `WalletAccount` extend it and `@Override kind()` (`"savings"` / `"wallet"`). `WalletAccount implements Payable` with `pay` that prints `Paid ₹` + amount and returns `true`. In `main`, store both in a `List<Account>`, loop and print `kind()` + holder (**no** `if (a instanceof ...)` in that loop). Then call `pay(120)` through a `Payable` variable on the wallet. No `throw`.

```
// your code
```

2. Write `InventoryLite` with `List<String> skuOrder`, `Set<String> knownSkus`, and `Map<String, Integer> prices` (SKU → INR). Methods: `add(sku, price)` rejects duplicate SKU and `price <= 0` with a printed message (`print` + `return`); `lookup(sku)` prints the price or `not found` (**no** crash). In `main`: add `P101` at `40`, add `P102` at `10`, add `P101` again, lookup `P102`, lookup `P999`. Left-hand types must be `List` / `Set` / `Map`. No `throw`.

```
// your code
```

C. Short answer (1)

In 3–6 sentences, explain when you choose a **List**, a **Set**, and a **Map** in a **Swiggy** canteen. Why keep a `List` of SKUs next to a `Map` of SKU → price instead of looping `HashMap.keySet()` for the receipt?

(Write 3–6 sentences.)